

Лабораторная работа №6 Автоматизация тестирования пользовательского интерфейса

Цель работы

Освоить методы автоматизированного тестирования пользовательского интерфейса с использованием библиотеки FlaUI для .NET приложений.

Задание

Для тест-кейсов из лабораторных 3 и 5 создать программный код для автоматизации тестирования в соответствии с вариантом.

Используемые инструменты и технологии

- Язык программирования: C#
- Фреймворк для тестирования: MSTest
- Библиотека для автоматизации UI: FlaUI

Пример выполнения задания

- Тестируемое приложение: ProductManager.exe

Исходный код тестов

```
// Атрибут, указывающий, что этот класс содержит тестовые методы
[TestClass]
public class ProductManagerTests
{
    // Поле для хранения экземпляра тестируемого приложения
    private Application _app;
    // Поле для автоматизации UI (используется библиотека FlaUI)
    private UIA3Automation _automation;
    // Поле для хранения главного окна приложения
    private Window _mainWindow;

    // Метод, выполняемый перед каждым тестом (атрибут TestInitialize)
    [TestInitialize]
    public void TestInitialize()
    {
        // Запуск приложения перед каждым тестом
        _app =
        Application.Launch(@"D:\Users\User\source\repos\productManager\ProductManager\bin\Debug\Produ
ctManager.exe");
    }
}
```

```

    // Инициализация автоматизации UI
    _automation = new UIA3Automation();
    // Получение главного окна приложения
    _mainWindow = _app.GetMainWindow(_automation);
}

// Метод, выполняемый после каждого теста (атрибут TestCleanup)
[TestCleanup]
public void TestCleanup()
{
    // Освобождение ресурсов автоматизации (если объект не null)
    _automation?.Dispose();
    // Закрытие приложения (если объект не null)
    _app?.Close();
}

// Тестовый метод для проверки добавления товара (атрибут TestMethod)
[TestMethod]
public void TestAddProduct()
{
    // Arrange (подготовка тестовых данных и окружения)

    // Поиск текстового поля для ввода имени товара по AutomationId
    var nameTextBox = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("nameTextBox")).AsTextBox();
    // Поиск элемента для ввода цены (NumericUpDown)
    var priceInput = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("numericUpDown1"));
    // Поиск внутреннего элемента NumericUpDown для установки значения
    var price = priceInput.FindFirstDescendant(cf => cf.ByName("Наименование товара"));
    // Поиск элемента для ввода количества (NumericUpDown)
    var quantityInput = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("numericUpDown2"));
    // Поиск внутреннего элемента NumericUpDown для установки значения
    var quantity = quantityInput.FindFirstDescendant(cf => cf.ByName("Вертушка"));
    // Поиск кнопки добавления товара
    var addButton = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("button1")).AsButton();

    // Act (выполнение тестируемого действия)

    // Установка текста в поле имени товара
    nameTextBox.Text = "Тестовый товар";
    // Установка значения цены через паттерн Value
    price.Patterns.Value.Pattern.SetValue("10");
    // Установка значения количества через паттерн Value
    quantity.Patterns.Value.Pattern.SetValue("5");
    // Клик по кнопке добавления
    addButton.Click();
    // Получение модального окна (ожидаем, что это окно сообщения)
    var messageBox = _mainWindow.ModalWindows.FirstOrDefault();
    // Поиск кнопки ОК в модальном окне (AutomationId "2" обычно соответствует ОК)

```

```

    var okButton = messageBox.FindFirstDescendant(cf => cf.ByAutomationId("2")).AsButton() ??
throw new Exception("кнопка не найдена.");
    // Клик по кнопке OK
    okButton.Click();

    // Assert (проверка результатов)

    // Получение списка товаров
    var productsList = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("productsList")).AsListBox();
    // Получение текста первого элемента списка
    var productText = productsList.Items.First().Text;
    // Проверка, что товар добавлен и строка имеет правильный формат
    Assert.IsTrue(productText.Contains("Товар: Тестовый товар, Цена: 10, Количество: 5"), "Товар
не был добавлен или формат строки неверный");
}

// Тестовый метод для проверки доступности товара
[TestMethod]
public void TestCheckProductAvailability()
{
    // Arrange

    // Сначала добавляем товар для проверки (используем ранее написанный тест)
    TestAddProduct();
    // Получение списка товаров
    var productsList = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("productsList")).AsListBox();
    // Получение первого товара из списка
    var product = productsList.Items[0];
    // Поиск кнопки проверки доступности
    var checkButton = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("button3")).AsButton();

    // Act

    // Клик по товару в списке
    product.Click();
    // Клик по кнопке проверки
    checkButton.Click();
    // Получение модального окна с сообщением
    var messageBox = _mainWindow.ModalWindows.FirstOrDefault();
    // Поиск текста сообщения (стандартный AutomationId 65535 для текста в MessageBox)
    var msgText = messageBox.FindFirstDescendant(cf => cf.ByAutomationId("65535"));
    // Получение текста сообщения
    string mText = msgText.Name;

    // Assert

    // Проверка, что сообщение содержит текст о доступности товара
    StringAssert.Contains(mText, "Товар доступен!", "Должен быть текст: Товар доступен!");
}

```

```

// Тестовый метод для проверки удаления товара
[TestMethod]
public void TestDeleteProduct()
{
    // Arrange

    // Сначала добавляем товар для удаления
    TestAddProduct();
    // Получение списка товаров
    var productsList = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("productsList")).AsListBox();
    // Поиск кнопки удаления
    var deleteButton = _mainWindow.FindFirstDescendant(cf =>
cf.ByAutomationId("button2")).AsButton();

    // Act

    // Выбор первого товара в списке
    productsList.Items[0].Select();
    // Клик по кнопке удаления
    deleteButton.Click();
    // Получение модального окна подтверждения
    var messageBox = _mainWindow.ModalWindows.FirstOrDefault();
    // Поиск кнопки ОК в модальном окне
    var okButton = messageBox.FindFirstDescendant(cf => cf.ByAutomationId("2")).AsButton();
    // Клик по кнопке ОК
    okButton.Click();

    // Assert

    // Проверка, что список товаров пуст (длина = 0)
    Assert.AreEqual(0, productsList.Items.Length, "Товар не был удалён");
}
}

```

Описание тестовых сценариев

1. Тест добавления товара (`TestAddProduct`)

****Цель:**** Проверить корректность добавления нового товара в список.

****Шаги:****

1. Заполнить поле "Наименование товара"
2. Установить цену товара
3. Установить количество товара
4. Нажать кнопку "Добавить"
5. Проверить наличие товара в списке

****Ожидаемый результат:**** Товар с указанными параметрами появляется в списке.

2. Тест проверки доступности товара (`TestCheckProductAvailability`)

****Цель:**** Проверить корректность работы функции проверки доступности товара.

****Шаги:****

1. Добавить тестовый товар
2. Выбрать товар в списке
3. Нажать кнопку "Проверить доступность"
4. Проверить текст сообщения

****Ожидаемый результат:**** Появляется сообщение "Товар доступен!".

3. Тест удаления товара (`TestDeleteProduct`)

****Цель:**** Проверить корректность удаления товара из списка.

****Шаги:****

1. Добавить тестовый товар
2. Выбрать товар в списке
3. Нажать кнопку "Удалить"
4. Подтвердить удаление
5. Проверить пустоту списка

****Ожидаемый результат:**** Список товаров пуст.

Вывод

В ходе лабораторной работы были освоены методы автоматизированного тестирования пользовательского интерфейса с использованием библиотеки FlaUI. Были реализованы тесты для проверки основных функций приложения управления товарами: добавление, проверка доступности и удаление товаров. Автоматизированные UI-тесты позволяют быстро и надежно проверять работоспособность приложения после внесения изменений.

Варианты

1. Управление банковским счётом

2. Конвертер валют

3. Управление задачами

4. Календарь событий

5. Управление книгами в библиотеке

6. Управление заметками

7. Управление контактами

8. Управление покупками

9. Управление задачами с приоритетом

- ### 10. Управление продажами
- ### 11. Управление инвентарём
- ### 12. Управление заказами
- ### 13. Управление сотрудниками
- ### 14. Управление проектами
- ### 15. Управление клиентами
- ### 16. Управление доставкой
- ### 17. Управление отчётами
- ### 18. Управление резервированием
- ### 19. Управление бюджетом
- ### 20. Управление файлами и папками
- ### 21. Управление системой безопасности данных
- ### 22. Управление здоровьем
- ### 23. Управление музыкальной коллекцией
- ### 24. Управление путешествиями
- ### 25. Управление фотографиями
- ### 26. Управление рецептами и планированием меню
- ### 27. Управление обучением и курсами
- ### 28. Управление резюме и поиском работы
- ### 29. Управление встречами и мероприятиями
- ### 30. Управление техникой и оборудованием
- ### 31. Управление автомобилем и его обслуживанием
- ### 32. Управление энергопотреблением
- ### 33. Управление личными целями и задачами
- ### 34. Управление коллекцией игр
- ### 35. Управление доставкой